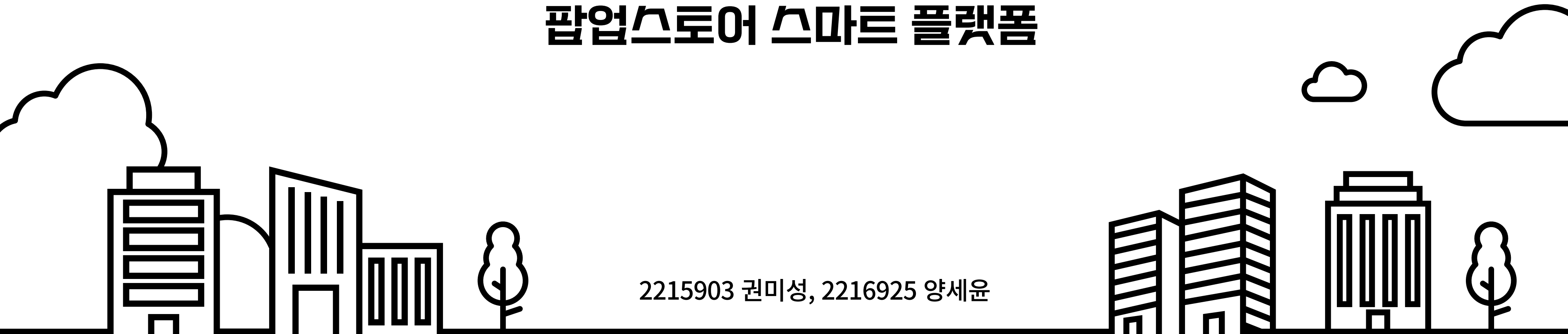


성공적인 브랜드를 만드는 전략

POP-IN

팝업스토어 스마트 플랫폼

2215903 권미성, 2216925 양세윤



서비스의 배경



**"짧고 강렬한 경험으로
소비자와 즉시 연결되는
시대, 팝업스토어는
브랜드의 새로운 무대가
되었다."**

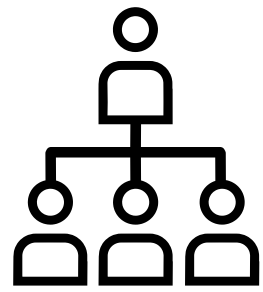
스마트 플랫폼은 팝업스토어의 운영 효율성과 고객 경험을 동시에 혁신합니다. 방문자 흐름을 실시간으로 관리하고, 맞춤형 정보를 자동 제공함으로써 브랜드와 소비자 간의 상호작용을 더 빠르고 강하게 연결합니다.

이러한 플랫폼은 운영자에게는 더 나은 매장 통제력을, 방문자에게는 더 높은 만족도를 제공합니다. 직접 줄서기 없는 입장, 즉각적인 체험 인증, 오픈채팅 등은 기존 오프라인 경험을 한층 업그레이드합니다. 결국, 스마트 플랫폼을 도입한 팝업스토어는 단순한 공간을 넘어 '경험 중심 마케팅의 중심지'로 거듭납니다.



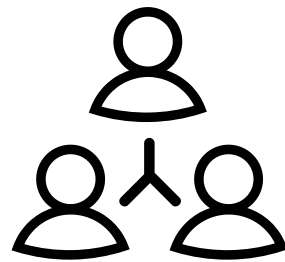
**팝업스토어를 스마트하게,
경험을 특별하게**

서비스 목표



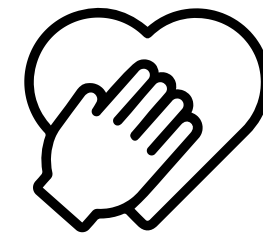
GOAL 1

매장 정보 / 이벤트
진행 정보 수신
(Push-to-Chat)



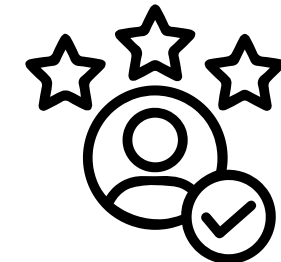
GOAL 2

자동연결 및 팝업 체험
인증 시스템
동일 부스 사용자 간
오픈채팅



GOAL 3

이용 시간 제한 /
체류 기록
피드백을 통한 운영
가능



GOAL 4

실시간 웨이팅 시스
템을 통한 입장 관리

Pop-in 프로토콜 주요 기능 및 사용방법

ID 기반 역할 분리 아키텍처

 관리자 (ID: 1-3)

1 ID 1-3 입력 → 2 자동 운영 시작 → 3 키보드 명령

m 메시지

s 상태

a 방송

q 조용히

t 타이머

w 대기열

 사용자 (ID: 4+)

1 ID 4+ 입력 → 2 자동 부스 탐색 → 3 부스 체험

y/n 체험 선택

c 채팅

e 퇴장

사용자 기능

자동 부스 탐색

RSSI 신호 강도 기반으로 최적의 부스를 자동 선택합니다.
2초 스캔 후 가장 가까우면서 여유있는 부스로 연결됩니다.

체험 관리

100초 세션 시간 제한으로 공정한 체험 기회를 제공합니
다. 1인 1회 체험 정책으로 중복 입장을 차단합니다.

대기열

실시간으로 대기 순번을 확인할 수 있습니다. 차례가 되면
자동으로 입장 알림을 받습니다.

채팅

Push-to-Talk 방식으로 부스 내 다른 사용자와 실시간 채
팅이 가능합니다. 'c' 키를 눌러 채팅 모드로 전환합니다.

조기 퇴장

체험 중 언제든지 'e' 키로 부스에서 나갈 수 있습니다. 대기
열에서도 동일한 키로 이탈 가능합니다.

관리자 기능

부스 운영

자동으로 부스 정보를 방송하여 사용자를 유치합니다.
Quiet 모드('q')로 자동 방송을 중지할 수 있습니다.

사용자 관리

세션 타이머로 각 사용자의 남은 시간을 관리합니다. 't'로
타이머 확인, 'w'로 대기열을 확인할 수 있습니다.

메시지 전송

'm' 키로 공지사항을 브로드캐스트할 수 있습니다. 사용자
간 채팅 메시지를 자동으로 중계합니다.

상태 모니터링

's' 키로 실시간 부스 현황을 확인할 수 있습니다. 현재 이용
자, 대기자, 총 등록자 수를 한눈에 파악합니다.

Pop-in 시스템 아키텍처

프로토콜 스택

L3 Layer

Pop-in 애플리케이션 로직
메시지 처리, FSM 관리, 세션 제어
사용자 인터페이스 및 비즈니스 규칙

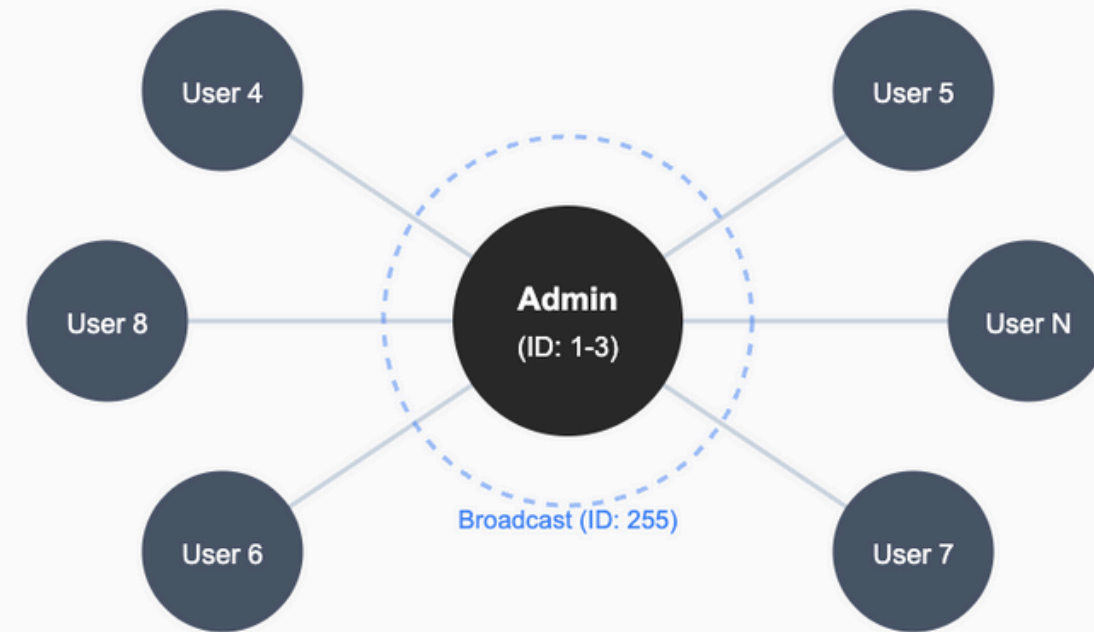
L2 Layer

Stop-and-Wait ARQ, 데이터 분할/조립
신뢰성 있는 데이터 전송 보장

PHY/MAC Layer

LoRa 무선 통신, RSSI/SNR 측정
물리적 신호 전송 및 수신

네트워크 토폴로지



일반 사용자

ID: 4 ~ 254

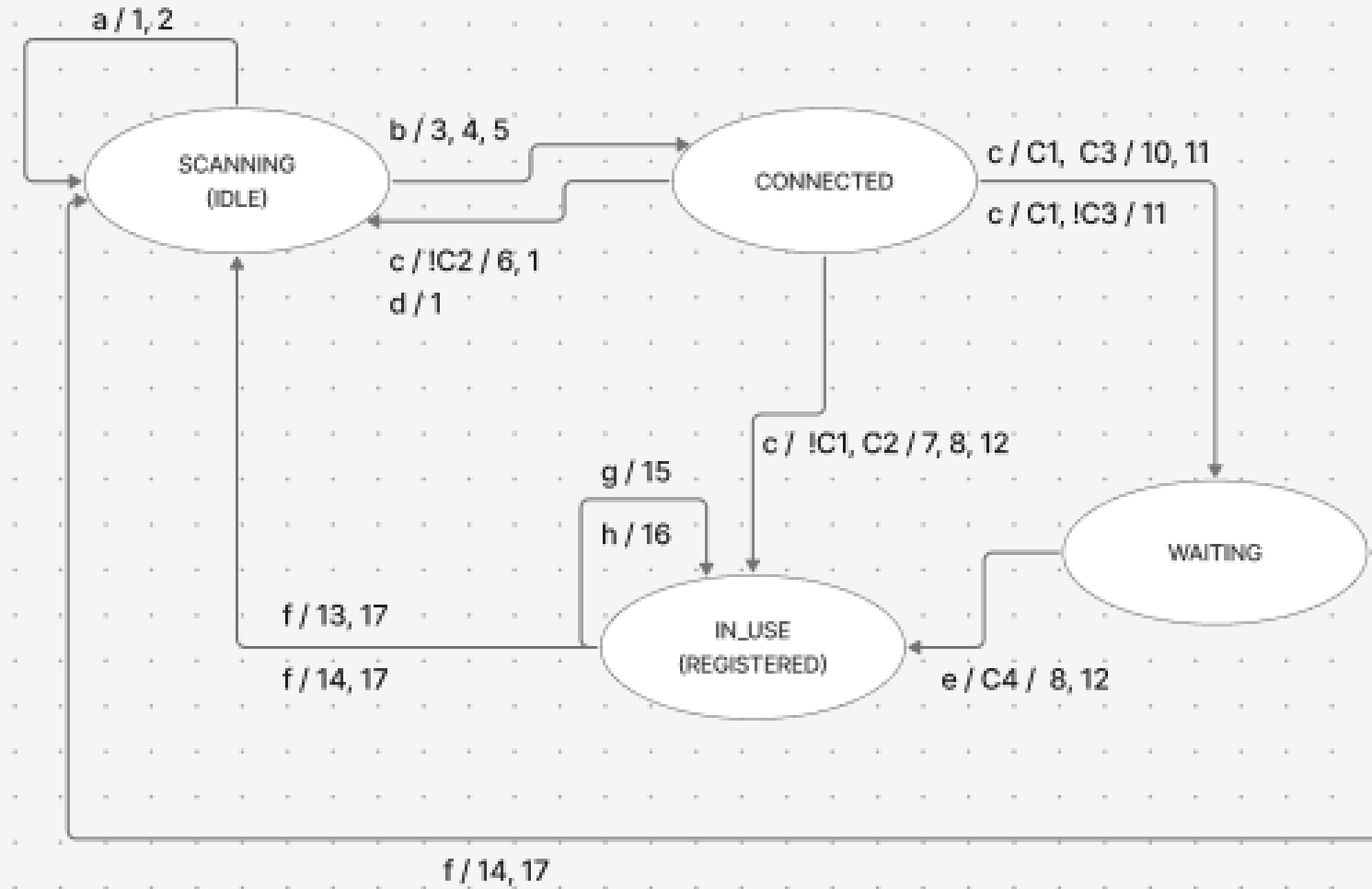
팝업스토어 방문객
자동 부스 탐색 및 체험

관리자

ID: 1 ~ 3

부스 운영자
사용자 관리 및 모니터링

FSM



사용자 정보 요청 → 가까운 버스 탐색, 연결 요청

Condition

- C1 : 현재 입장 인원 초과 (capacity full)
→ !C1 : 입장 가능 상태
- C2 : userID ≠ registeredList
→ !C2 : userID ∈ registeredList (이미 등록됨)
- C3 : userID ≠ waitingQueue
→ !C3 : userID ∈ waitingQueue (웨이팅 이미 등록됨)
- C4 : nextUser == UserID

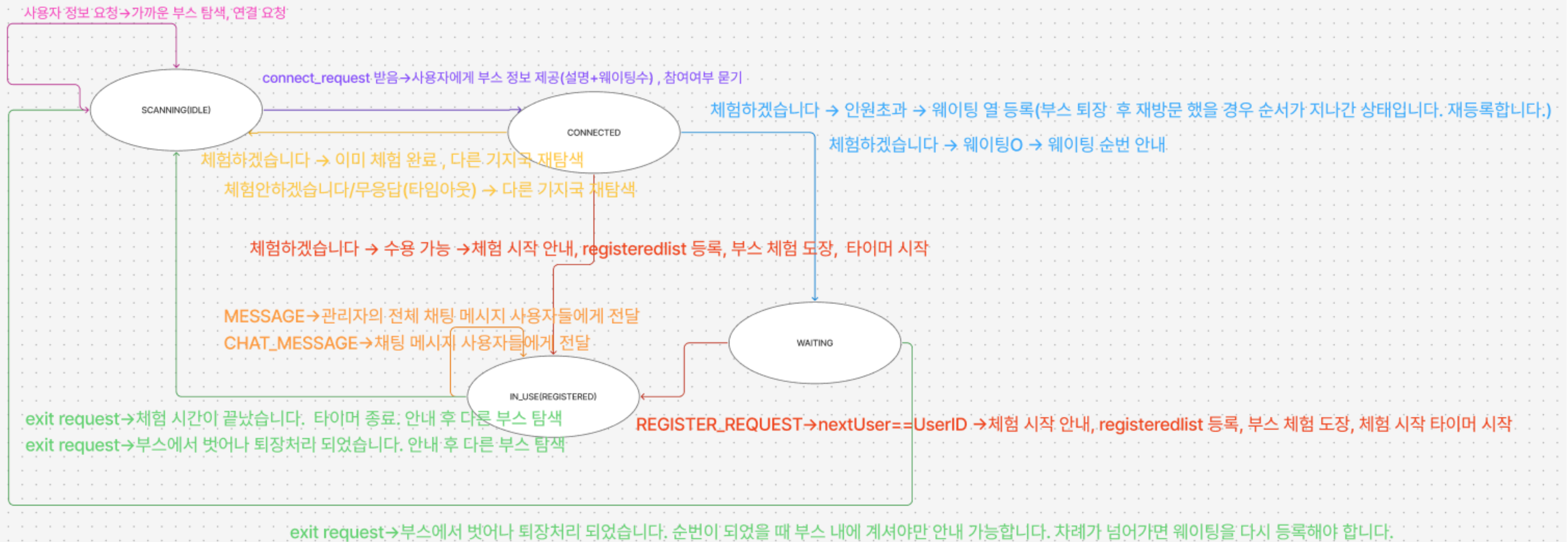
Event

- a. USER_INFO_REQUEST
- b. CONNECT_REQUEST 받음
- c. USER_RESPONSE == YES
- d. USER_RESPONSE == NO
- e. REGISTER_REQUEST 받음
- f. EXIT_REQUEST 받음
- g. MESSAGE 받음 (공지, 알림 등)
- h. CHAT_MESSAGE 받음 (사용자 채팅)

Action

- 부스 리스트 정렬 → 최적 부스 연결 요청
- CONNECT_REQUEST 전송
- 연결 확정 후 상태 업데이트
- 사용자에게 부스 정보 제공(부스 설명 및 현재 이용자 수, 웨이팅 인원 등)
- 사용자에게 참여여부 묻기("이 부스를 체험하시겠습니까?(Y/N)")
- 등록된 사용자 입장 거부 및 알림 전송 ("체험은 1회만 가능합니다")
- 등록 요청 (REGISTER_REQUEST) 전송
- 등록 성공 처리 및 알림 (부스 체험 시작 알림, registeredList와 activeList 등록, 체험 도장)
- 등록 실패 처리 및 재시도
- 대기열 요청 (QUEUE_JOIN_REQUEST) 전송
- 대기 번호 표시 (QUEUE_INFO)
- 세션 시작, 타이머 시작
- 타이머 만료 → TIMEOUT_ALERT 송신, activeList에서 삭제
- 사용자 자발적 퇴장 처리 (퇴장 처리, 이용시간 기록, 리스트 갱신 등) → activeList에서 삭제
- 메시지 수신 시 사용자에게 전달 (공지, 관리자 메시지 등)
- 채팅 메시지 수신 시 체험 사용자에게 전달
- 다른 base station 감지 시 재스캔 시작 (registeredList 초기화 없이)

FSM



RSSI 기반 부스 선택 알고리즘

실시간 부스 평가

Booth A

-30점

신호 강도

-50 dBm

수용 현황

1/2

대기 인원

0명

계산: $-50 + 20 - 0 = -30$

✓ 최적 선택

Booth B

-60점

신호 강도

-40 dBm

수용 현황

2/2

대기 인원

2명

계산: $-40 + 0 - 20 = -60$

Booth C

-35점

신호 강도

-45 dBm

수용 현황

1/2

대기 인원

1명

계산: $-45 + 20 - 10 = -35$

$$\text{최종 점수} = \text{RSSI} + \text{빈자리 보너스}(+20) - \text{대기인원} \times 10$$

선택 알고리즘 프로세스

1

모든 부스에
스캔 요청 전송

2

RSSI 및 상태
정보 수집 (2초)

3

점수 계산 및
순위 결정

4

최고 점수 부스
자동 선택

RSSI (Received Signal Strength Indicator)

- 수신 신호 강도를 dBm 단위로 측정
- 30 dBm (매우 강함) ~ -90 dBm (매우 약함)
- 가까운 부스일수록 높은 신호 강도
- 주요 평가 지표로 활용

점수 계산 방식

- 기본 점수: RSSI 값 (음수)
- 빈자리 보너스: +20점 (여유 공간 존재시)
- 대기 페널티: -10점 × 대기 인원수
- 최종 점수: 높을수록 (0에 가까울수록) 우선

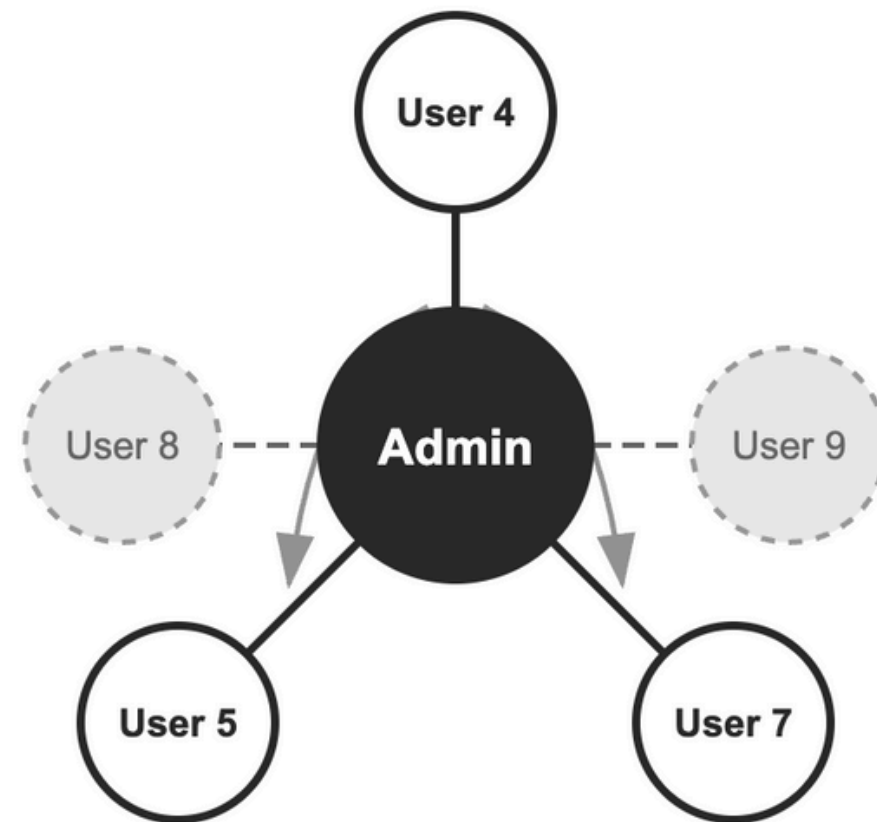
알고리즘 특징

- 신호 강도와 혼잡도를 동시 고려
- 2초 스캔으로 신속한 결정
- 자동 최적화로 사용자 편의성 제공
- 공정한 부스 배분 실현

Push-to-Talk 채팅 시스템

Star Topology 중계 방식

Push-to-Talk 채팅 시스템



활성 사용자: 부스 내 체험 중

비활성 사용자: 대기 중



키 기반 활성화

'c' 키로 채팅 모드
전환



Star Topology

관리자가 모든 메
시지 중계



부스별 격리

같은 부스 사용자
끼리만



발신자 ID 보존

투명한 메시지 전
달

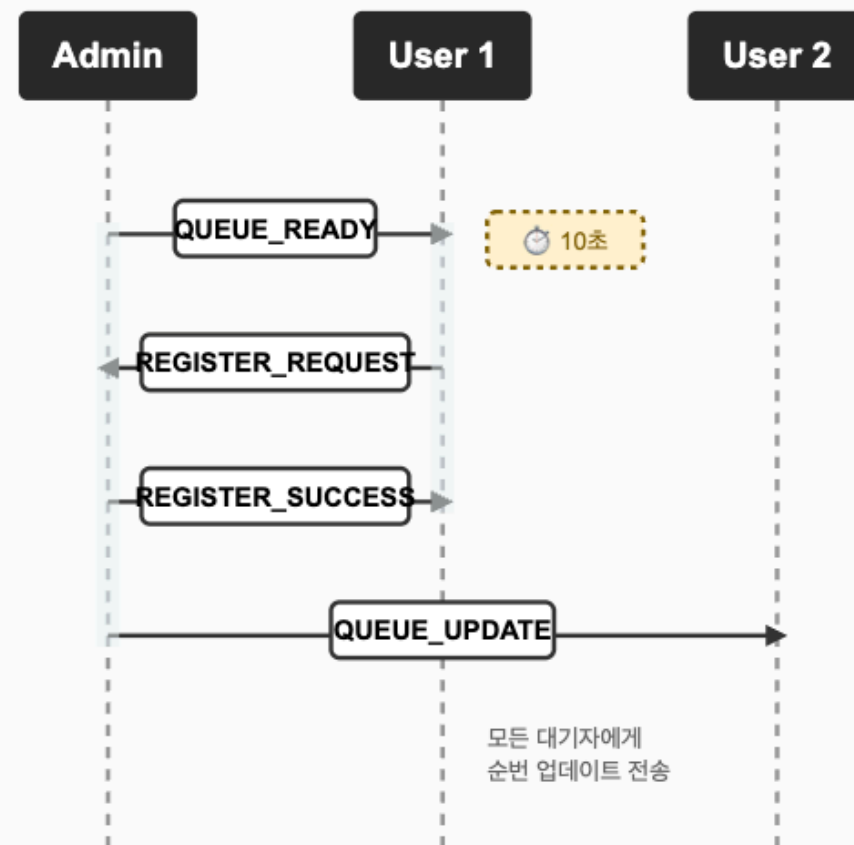
메시지 흐름

User4 → Admin → User5, User7

(발신자 ID 포함하여 중계)

Pull 기반 대기열 시스템

메시지 시퀀스 다이어그램



1 서버 주도형 호출

관리자가 입장 시점을 제어하여 부스 혼잡을 방지합니다

2 위치 기반 검증

10초 내 부스 근처에서 응답해야 입장 가능합니다

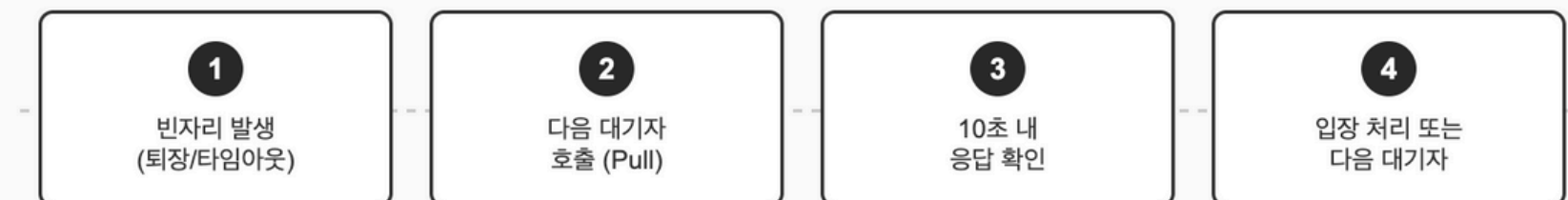
3 자동 스킵 기능

무응답 시 다음 대기자로 자동으로 넘어갑니다

4 실시간 업데이트

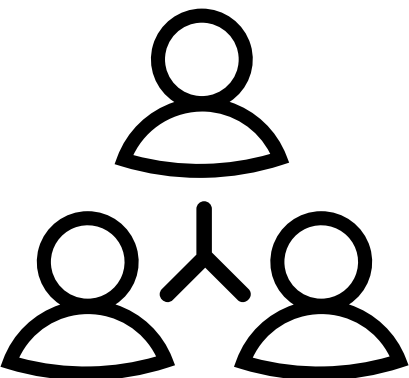
순번 변경 시 모든 대기자에게 즉시 알림을 전송합니다

대기열 처리 플로우





사용자 시나리오



시나리오 1 :기본 부스 체험

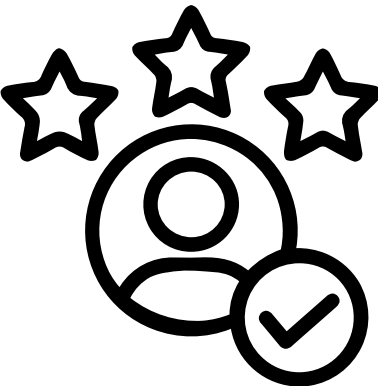
```
[User 5]                                [Admin 1 - Booth 1]
|                                         |
| (자동 RSSI 스캔 시작)                 | (부스 운영 중)
|----BOOTH_SCAN---->                   |
|<---BOOTH_ANNOUNCE(RSSI:-45dBm)-----|
|                                         |
| (2초 후 Booth 1 자동 선택)             |
|----CONNECT_REQUEST---->              |
|<---BOOTH_INFO(0/2, "Pop-up Store")-   |
|                                         |
| "Would you like to experience? y"     |
|----USER_RESPONSE(YES)---->           |
|----REGISTER_REQUEST---->             |
|<---REGISTER_RESPONSE(SUCCESS)-----|
|                                         |
| (100초 체험 시작)                    |
| Time remaining: 95 seconds             |
|                                         |
| (100초 후)                           |
|<---TIMEOUT_ALERT-----|
| "Session expired!"                   |
| (자동으로 SCANNING 복귀)              |
```

시나리오 2 :채팅 기능

```
[User 8]                                [Admin 3]                                [User 9]
| (IN_USE 상태)                        |                                         | (IN_USE 상태)
|                                         |                                         |
| Press 'c'                            |                                         |
| "Hello everyone!"                    |                                         |
|--CHAT_MESSAGE----->|                                         |
|                                         |--CHAT_MESSAGE----->|
|                                         | (senderId: 8) |
|                                         |                                         |
|                                         | "[CHAT]User 8:
|                                         | Hello everyone!"
|                                         |
|                                         |<-CHAT_MESSAGE-----|
|                                         | "Hi User 8!" |
|<-CHAT_MESSAGE-----|                                         |
| "[CHAT] User 9: Hi User 8!"         |
```



사용자 시나리오



시나리오 3 : 대기열 관리

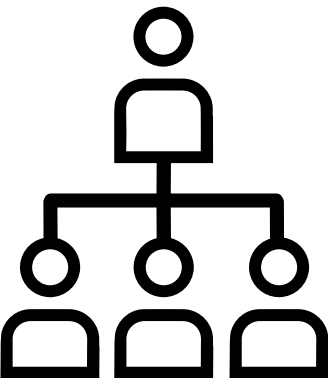
[User 6]	[User 7]	[Admin 2 - Booth 2]
		(2/2 만석)
--REGISTER_REQUEST-->		
<-REGISTER_RESPONSE--		
(FULL_WAITING)		
<-QUEUE_INFO(1/1)-----		
	--REGISTER_REQUEST----->	
	<-REGISTER_RESPONSE-----	
	(FULL_WAITING)	
	<-QUEUE_INFO(2/2)-----	
<-QUEUE_UPDATE(1/2)--		
	(User 4 퇴장)	
		--EXIT-->
<-QUEUE_READY-----		
"Your turn!"		
--REGISTER_REQUEST-->		
<-REGISTER_RESPONSE--		
(SUCCESS)		
	<-QUEUE_UPDATE(1/1)-----	

시나리오 4 : 대기열 타임아웃

[User 12 - 원거리]	[User 13 - 대기중]	[Admin 1]
	(Position: 2/2)	
	(User 체험 종료)	
	<---QUEUE_READY-----	
	(10초 응답 시간)	
	(부스에서 멀어 응답 못함)	
	<---QUEUE_UPDATE(0/0)-----	
	"Removed from queue"	
(대기 중 Position: 1/1)		
<---QUEUE_READY-----		
---REGISTER_REQUEST----->		
<---REGISTER_RESPONSE-----		
(입장 성공)		



사용자 시나리오



시나리오 5 : 재입장 차단

```
[User 10]                                [Admin 1]
| (이전에 Booth 1 체험 완료)              | (registeredList에
|                                         | User 10 존재)
|
|-----CONNECT_REQUEST---->
|<---BOOTH_INFO-----|
| "Would you like to experience? y"|
|-----REGISTER_REQUEST--->
|<---REGISTER_RESPONSE-----|
|      (ALREADY_USED)
| "You have already experienced!"
| (SCANNING으로 복귀)
```

시나리오 6 : 관리자 브로드캐스트

```
[Admin 1]                                [User 14]                                [User 15]
| (IN_USE: User 14, 15)                  | (체험 중)                          | (체험 중)
|
| Press 'm'
| "Special event at 3PM!"
|-----ADMIN_MESSAGE----->|
|-----ADMIN_MESSAGE----->|
|
|                                     |
|                                     | "[ADMIN BROADCAST]"
|                                     | Special event at 3PM!"
|
| "2 users received"
|
```



사용자 시나리오



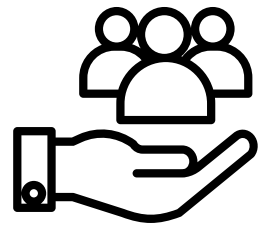
시나리오 7 : RSSI 기반 부스 선택

```
[User 11]
|
| Starting RSSI-based scan...
|----BOOTH_SCAN----> Admin 1,2,3
|
|<---BOOTH_ANNOUNCE--- Admin 1 (RSSI: -65dBm, Users: 2/2, Queue: 3)
|<---BOOTH_ANNOUNCE--- Admin 2 (RSSI: -45dBm, Users: 1/2, Queue: 0)
|<---BOOTH_ANNOUNCE--- Admin 3 (RSSI: -40dBm, Users: 2/2, Queue: 1)
|
| Analyzing booth options...
| Booth 1: RSSI=-65, Score=-95 (Available=0/2, Queue=3)
| Booth 2: RSSI=-45, Score=-25 (Available=1/2, Queue=0) ← 최고 점수
| Booth 3: RSSI=-40, Score=-50 (Available=0/2, Queue=1)
|
| Selected Booth 2 as optimal choice
|----CONNECT_REQUEST----> Admin 2
```

시나리오 8 : 네트워크 장애 복구

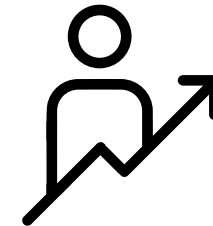
```
[User 16]                                [Admin 1]
|                                           |
|----CONNECT_REQUEST---->                |
|           (패킷 손실)                    |
| (3초 타임아웃)                          |
| "No response, Retrying (1/3)"            |
|----CONNECT_REQUEST---->                |
|<---BOOTH_INFO----->                  |
|                                           |
|----REGISTER_REQUEST---->                |
|           (L2 ARQ 재전송 x3)              |
|<---REGISTER_RESPONSE----->            |
| (정상 등록)                              |
```

결과물에 대한 기대효과



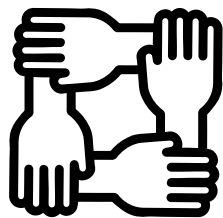
운영 인력 최소화

웨이팅 등록, 고객 관리, 체류시간 측정 등을 자동화하여 운영 인력을 최소화



고객 만족도 향상

앱 설치 없이 자동 연결되고 실시간 안내로 고객 만족도가 증가되며 혼잡한 공간에서의 스트레스 감소, 브랜드에 대한 긍정적 인식 유도



향후 운영 전략 수립 및 타겟 마케팅

고객 방문 시간, 체류 시간, 웨이팅 시간 등 사용자 통계를 확보하여 향후 운영 전략 수립



운영 효율성 향상

단기/이동형 매장에서 효율적인 인원 통제 및 서비스 제공이 가능하여 운영 효율성 향상

활용방안

단기/소규모 이벤트에서도 높은 확장성과 적용 편의성을 확보

다양한 팝업스토어

패션/뷰티 브랜드, 푸드트럭, 전시형 이벤트 등

공공 서비스

학교 행사 부스, 체험 부스, 지역 커뮤니티 네트워크, 병원, 박람회 등

무인 매장, 키오스크

자동으로 설명 제공, 소통 가능으로 무인매장, 스마트 부스, 키오스크 기반 시스템 등

질의 응답 및 연락처

자유롭게 질문해주세요.



이메일

michel1612@sookmyung.ac.kr

esuny12@sookmyung.ac.kr

더 궁금하신 점은 위 연락처를 참고해주세요.

감사합니다

한 학기동안 다들 고생하셨습니다.